

迦密聖道中學
2024 - 2025 年度中五級上學期考試

資 訊 及 通 訊 科 技 科

試 題 答 題 簿

日期：二零二五年一月十五日
時間：上午八時十五分至上午九時四十五分
時限：一小時三十分鐘完卷
頁數：16 頁
總分：80

考生須知：

- 1. 宣布開考後，考生須首先在第一頁之適當位置填寫個人資料。
- 2. 本卷分甲、乙兩部份。各部必須用中文全答。
- 3. 考生須在本卷指定位置上作答。寫於指定位置以外的答案，將不予評閱。

姓名	
班別	
學號	
試場	

由閱卷員填寫

甲 1	/ 9
甲 2	/ 4
甲 3	/ 4
甲 4	/ 8
甲 5	/ 17
乙 1	/ 6
乙 2	/ 8
乙 3	/ 10
乙 4	/ 14
總分	/ 80

甲部（42 分）：試算表及數據庫

- 以下是一個包含兩個工作表的試算表，分別為 Marks 和 Summary，由陳老師用來記錄學生的測驗成績。Marks 的數據按不同科目進行分組。部分的數據如下：

Marks

	A	B	C	D	E	F
1	STUID	NAME	CLASS	SUBJECT	MARK	
2	S2017004	Anson	5A	Chinese	78	
3	S2017008	Edan	5A	Chinese	87	
4	S2017012	Frankie	5A	Chinese	41	
5	S2017016	Thomas	5B	Chinese	86	
.	.	.	.	.	.	
121	S2017076	Stanley	5D	Chinese	71	
122	S2017004	Anson	5A	English	57	
123	S2017008	Edan	5A	English	55	
124	S2017012	Frankie	5A	English	97	
125	S2017016	Thomas	5B	English	83	
.	.	.	.	.	.	
241	S2017076	Stanley	5D	English	63	
242	S2017004	Anson	5A	Maths	74	
243	S2017008	Edan	5A	Maths	77	
244	S2017012	Frankie	5A	Maths	94	
245	S2017016	Thomas	5B	Maths	67	
.	.	.	.	.	.	
361	S2017076	Stanley	5D	Maths	62	

陳老師利用 Summary 工作表整理各同學的分數，以方便檢閱，部分的數據如下：

Summary

	A	B	C	D	E
1	STUID	Chinese	English	Maths	TOTAL
2	S2017004				
3	S2017008				
4	S2017012				
5	S2017016				
.	.				
121	S2017076				

- (a) (i) 為了把學生的測驗成績整理到工作表 Summary，陳老師在工作表 Marks 中 F2 的儲存格輸入了公式 =A2&D2，然後複製到 F2:F361。請寫出 F361 的值。

(1 分)

- (ii) 在工作表 Summary 中，陳老師在 B2 中輸入公式，然後複製到 B3:D121，以顯示各同學的分數。寫出 B2 的公式。

=XLOOKUP (_____ , _____ , _____)

(3 分)

- (b) 在 Summary 中，為顯示分數的總和，在 E2 中輸入公式，然後複製到 E3:E121。寫出 E2 的公式。

(2 分)

- (c) 然而，甘老師使用了試算表中的樞紐分析表，可以輕鬆生成如下所示的表格，類似於 Summary 工作表：

樞紐分析表

	A	B	C	D	E
1	STUID	Chinese	English	Maths	GRAND TOTAL
2	S2017004	78	57	74	209
3	S2017008	87	55	77	219
4	S2017012	41	97	94	232
5	S2017016	86	83	67	236
.	.	.	.	.	.
.	.	.	.	.	.
121	S2017076	71	63	62	196

完成下列部份以建構樞紐分析表

	欄

列	值
_____	_____ 的 _____

(3 分)

2. 小明為圖書館設計了一個管理運作的數據庫系統。數據庫設計需滿足以下要求：

- 圖書館有多名會員，每名會員可以借閱多本書籍。
- 每本書可以被多名會員借閱，但每次借閱記錄僅限於一名會員。
- 借閱記錄用於連接會員和書籍，每條記錄代表一次借閱行為。

請根據上述要求完成下列數據庫的實體關係圖。在圖內只需繪製實體與關係，不需要標註屬性。



(4 分)

3. 數據表 PRODUCT 儲存產品資料，其結構如下：

欄名	描述	例子
PID	產品編號	P1001
Name	產品名稱	筆記本電腦
Category	產品類別	電子產品
Price	價格（港幣）	8999

主關鍵碼：PID

公司銷售按類型分為以下三類：

- 個人消費
- 公司採購
- 批發

每位顧客提交的訂單包含了他們購買的產品總結報告，如以下的報告樣本所示：

產品銷售報告

顧客名稱：陳小明				
產品類別：公司採購				
日期	產品名稱	數量	價格	總額
22/11/2024	筆記本電腦	2	8999	17998
23/11/2024	滑鼠	5	199	995

根據 PRODUCT 表和報告樣本內的資料，建立一個符合第三範式的數據庫模式。指出相應的主關鍵碼和外鍵碼，如不適用，請寫上 N/A。請以底線標示主關鍵碼。

Customer (_____ , _____) 外鍵碼： _____

Order (_____ , _____ , _____) 外鍵碼： _____

Product (_____ , _____ , _____ , _____) 外鍵碼： _____

OrderDetails (_____ , _____ , _____ , _____) 外鍵碼： _____

(4 分)

4. 小華正在設計一個數據庫，用來管理學生的基本資料。相關需求如下：

- 每位學生有唯一的學生編號 (SID)，作為主關鍵碼。
- 學生的資料包括名字 (NAME)、出生日期 (DOB) 和年齡 (AGE)。

(a) 寫出 SQL 命令來建立一個名為 STUDENTS 的數據表，並將 SID 設為主關鍵碼。

(2 分)

(b) 他需要在資料表中新增一位名為 "Mary"，出生日期為 2008 年 1 月 15 日，年齡為 16 歲，學生編號為 "S001" 的學生。請寫出一條 SQL 語句，將下列學生資料新增至 STUDENTS 表中。

(2 分)

(c) 若發現新輸入的學生資料有誤（如去年資料的重複輸入），應如何修正？請簡述操作方法及其目的。

(2 分)

(d) (i) 解釋為何 AGE 是一個衍生屬性。

(1 分)

(ii) 在數據表中使用 AGE 有什麼好處？舉出一個原因並加以說明。

(1 分)

5. 一家醫院管理系統包含以下三個數據表：

DOCTOR

欄名	類型	描述	例子
DoctorID	VARCHAR(4)	醫生編號（主鍵）	D001
Name	VARCHAR(50)	醫生姓名	陳大文
Specialty	VARCHAR(50)	專科	內科

PATIENT

欄名	類型	描述	例子
PatientID	VARCHAR(4)	病人編號（主鍵）	P001
Name	VARCHAR(50)	病人姓名	李小明
Age	INT	年齡	65

APPOINTMENT

欄名	類型	描述	例子
AppointmentID	VARCHAR(4)	預約編號（主鍵）	A001
DoctorID	VARCHAR(4)	醫生編號（外鍵，連接 DOCTOR）	D001
PatientID	VARCHAR(4)	病人編號（外鍵，連接 PATIENT）	P001
Date	DATE	預約日期	2024-12-01

- (a) 計算每位醫生的總預約數，並按預約數量從高到低排序。

(3 分)

- (b) 查詢那些超過 60 歲的病人編號及其最後一次預約日期。

(4 分)

- (c) 查找沒有任何預約的醫生姓名。

(3 分)

- (d) 創建一個名為 `DoctorPatientSummary` 的檢視，其中包含醫生的姓名、專科，以及每位醫生的病人數量。

(4 分)

- (e) 列出所有病人和醫生的姓名，並在結果中區分“角色”是病人還是醫生。

(3 分)

乙部 (38 分)：算法與程式編寫

1. 陳老師建立了兩個隊列 Q1 和 Q2。以下是一些已定義的子程序：

子程序	描述
enqueue(Q, K)	將項目 K 加入到隊列 Q 的末尾。
dequeue(Q)	移除並傳回隊列 Q 的第一個項目，如果 Q 為空，則傳回 -1。
isEmpty(Q)	如果隊列 Q 為空，傳回 TRUE；否則傳回 FALSE。

- (a) 假設 Q1 和 Q2 在開始時均為空。請寫下執行以下偽代碼後，Q1 和 Q2 的最終內容。

```
enqueue(Q1, 3)
enqueue(Q1, 9)
enqueue(Q1, 12)
當 NOT isEmpty(Q1) 執行
    enqueue(Q2, dequeue(Q1))
enqueue(Q2, 15)
```

	首					末
Q1:						
	首					末
Q2:						

(2 分)

陳老師使用 Q1 和 Q2 實現了堆疊 S。以下是新增的兩個子程序：

子程序	描述
pop()	彈出並傳回堆疊 S 的頂部元素，如果 S 為空，則傳回 -1。
push(K)	將項目 K 推入堆疊 S 的頂部。

- (b) 如果 Q1 的首部是堆疊 S 的頂部，並且 Q1 的尾部是堆疊的末部，完成以下 pop() 及 push(K) 子程序的偽代碼：

子程序 pop()

子程序 push(K)

```
當 NOT isEmpty(Q1) 執行
    enqueue(Q2, )
```

```
當 NOT isEmpty(Q2) 執行
```

(4 分)

2. 中綴表達式 是一種運算表達式，其中所有的運算符位於其操作數之間，我們在數學學習的表達式。
後綴表達式 是另一種運算表達式，其中所有的運算符位於操作數之後。

例子：

中綴表達式	後綴表達式	運算結果
$2 + 3$	$23+$	5
$(2+3) * (4+5)$	$23+45+*$	45
$2 + 3 * 4 + 5$	$234*+5+$	19

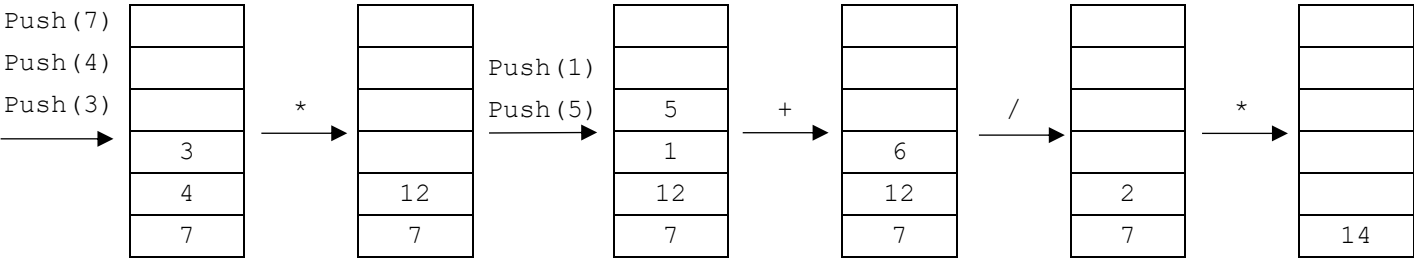
此外，由於我們的四個運算符是左結合的，表達式 $2+3+4$ 會被轉換為 $23+4+$ ，而不是 $234++$ 。

(a) 計算下列後綴表達式的值：

(i) $21-3+$ _____

(ii) $4321+-*$ _____ (2 分)

陳老師使用堆疊來計算後綴表達式的值。例如，要計算 $743*15+/*$ ，計算過程如下：



以下是陳老師使用的子程式：

子程式	描述
Push (X)	將項目 x 推入堆疊的頂部。
Pop ()	從堆疊的頂部彈出一個項目。

(b) 完成以下的子程式 EP (A) 的偽代碼計算後綴表達式的值，其中陣列 A 是存儲後綴表達式的陣列：

```

EP (A)
  設 i 由 1 至 (陣列 A 的大小) - 1
  ch ← A[i]
  如果 ch 是一個數字 則
    [ ]
  否則
    Num1 ← [ ]
    Num2 ← [ ]
    如果 content = "+" 則
      Push ( [ ] )
    否則 如果 content = "-" 則
      [ ]
    否則 如果 content = "*" 則
      [ ]
    否則 如果 content = "/" 則
      [ ]
  傳回 Pop ()
  
```

(6 分)

3. 一名學生設計了一個程式，用於找出兩個陣列 A 和 B 中的所有重複整數，並將這些重複的整數存儲到另一個堆疊 C 中。

例如，當陣列 A 和陣列 B 的內容如下：

陣列 A	0	1	2	3	4	5	...
	5	98	67	91	47	39	

陣列 B	0	1	2	3	4	5	...
	68	46	47	98	84	81	

以下是程式運行後堆疊 C 的內容：

堆疊 C	0	1	...
	98	47	

以下子程式可以在該程式中使用：

子程式	描述
LEN(A)	返回陣列 A 中的項目數量。
SORT(A)	使用快速排序法對陣列 A 按升序進行排序。
PUSH(S, V)	將數值 V 添加到堆疊 S 的末尾。

- (a) 完成以下偽代碼，使程式能夠輸出預期的結果。：

偽代碼

```

I ← 0
當 I < LEN(A) 執行
    J ← 0
    當 J < LEN(B) 執行
        如果  則
            
        J ← J + 1
    I ← I + 1

```

(2 分)

- (b) (i) 陳老師提議可以把其中一個陣列排序，可以使其效率提高。為什麼？

(2 分)

- (ii) 完成以下偽代碼，使程式能夠輸出預期的結果。

偽代碼

```
SORT(B)
I ← 0
當 I < LEN(A) 執行
    First ← 0
    Last ← 
    WHILE  DO
        Middle ← (First + Last) / 2
        IF  THEN
            PUSH(C, A[I])
        ELSE IF  THEN
            First ← Middle + 1
        ELSE
            Last ← Middle - 1
    I ← I + 1
```

(4 分)

- (iii) 當陣列 A 的大小為 16，陣列 B 的大小為 1024，使用快速排序法排序時，比較的次數分別如下：

陣列 A：需要進行比較 64 次

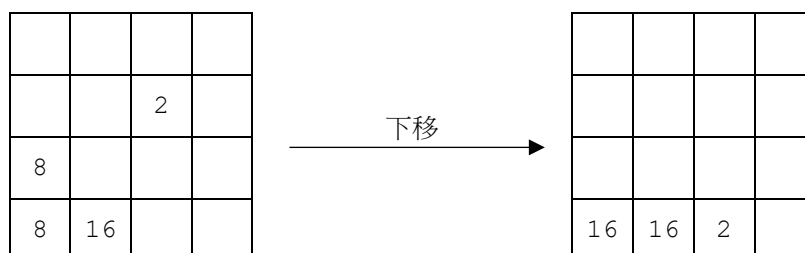
陣列 B：需要進行比較 10240 次

我們應該排序陣列 A 還是陣列 B，使執行程式 (b)(ii) 的效率更高呢？試簡略說明。

(2 分)

4. 2048 是一款受歡迎的單人益智遊戲，於 2014 年首次發布。該遊戲在一個 4x4 的方格網上進行，每個方格可能的數值包括：2、4、8、16、32、64、128、256、512、1024 或 2048。

遊戲的目標是將具有相同數值的方塊合併，生成更高數值的方塊，最終目標是創建一個數值為 2048 的方塊。玩家可以選擇將方塊向四個方向移動（上、下、左或右），每次移動會使所有方塊朝該方向滑動，直到它們到達網格的邊緣或與另一個方塊相撞。當兩個具有相同數值的方塊相撞時，它們會合併成一個新方塊，其數值為兩個方塊數值的總和。以下展示了一個移動的示例：



遊戲結束的條件有兩種：

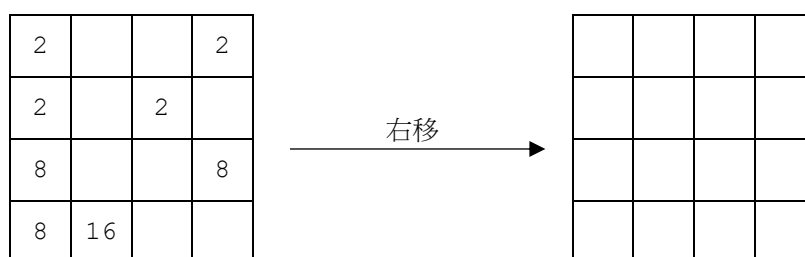
1. 當玩家創建了一個數值為 2048 的方塊時，遊戲結束。
2. 當再也無法進行任何移動時（即，網格已被方塊完全填滿，且沒有相鄰的方塊具有相同數值），遊戲結束。

例如：

2	4	2	4
4	2	8	2
2	8	4	8
8	4	2	4

在這個例子中，由於沒有相鄰的方塊具有相同數值可供合併，因此無法再進行任何移動，遊戲結束。

- (a) 請參考下面的遊戲板。在玩家將方塊向右移動後，填寫遊戲板中方塊的數值。



(1 分)

一個二維列表用於定義 2048 遊戲的遊戲板，如下所示：

```
board = [
    [0, 0, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0]
]
```

遊戲板列表初始化為 4 行 4 列，每個位置的初始值為 0，表示一個沒有方塊的空棋盤。

- (b) 每次玩家移動後，遊戲板上會新增一個數值為 2 或 4 的新方塊，其中有 90% 的機率生成數值為 2 的方塊，10% 的機率生成數值為 4 的方塊。新方塊會隨機添加到棋盤上的一個空格中。

完成以下 Python 程式中的 `add_tile()` 函數，用於在遊戲板上添加新方塊。

行數 程式碼

```

1 def add_tile():
2     empty = []
3     for i in range(4):
4         for j in range(4):
5             if :
6                 empty.append([i, j])
7     x, y = random.choice(empty)
8     if random.random() < 0.9:
9         
10    else:
11        

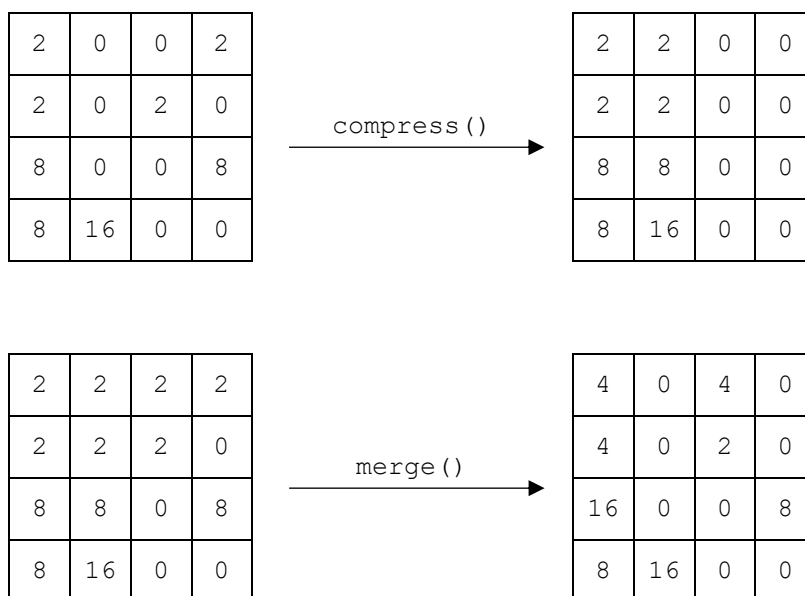
```

(2 分)

- (c) 以下是一些函數的描述：

函數	描述
<code>compress()</code>	將數值靠左對齊。
<code>merge()</code>	合併行中相鄰且相同的數值，合併後左側的值翻倍，右側設為零。

例子：



`moveLeft()` 函數用於將遊戲板上的方塊向左移動。該函數的實現如下所示：

行數 程式碼

```

1 def moveLeft():
2     compress()
3     merge()
4     compress()

```

(i) 完成以下 Python 程式中的 compress() 函數。

```
def compress():
    global board
    for i in range(4):
        new_row = []
        for j in range(4):
            if [ ]:
                new_row.append([ ])
        while len(new_row) < len(board[i]):
            new_row.append(0)
        board[i] = new_row
```

(2 分)

(ii) 完成以下 Python 程式中的 merge() 函數。

```
def merge():
    global board
    for i in range(4):
        for j in range(3):
            if board[i][j] != 0 and [ ]:
                [ ]
            board[i][j + 1] = 0
```

(2 分)

(d) 為了完成向上、向右及向下移動，定義了以下函數：

函數	描述
reverse()	反轉每行。
transpose()	矩陣轉置。

以下是 reverse() 函數的 Python 程式碼。

```
def reverse():
    new_board = []
    for i in range(len(board)):
        reversed_row = []
        for j in range(len(board[i]) - 1, -1, -1):
            reversed_row.append(board[i][j])
        new_board.append(reversed_row)
    board = new_board
```

以下是 transpose() 函數的 Python 程式碼。

```
def reverse():
    new_board = [
        [0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0],
        [0, 0, 0, 0]
    ]
    for i in range(len(board)):
        for j in range(len(board[0])):
            new_board[j][i] = board[i][j]
    board = new_board
```

(i) 完成以下 `moveRight()` 函數，以完成向右移動。

```
def moveRight():
```

```
    moveLeft()
```

(2 分)

(ii) 完成以下 `moveUp()` 函數。

```
def moveUp():
```

```
    moveLeft()
```

(2 分)

(iii) 完成以下 `moveDown()` 函數。

```
def moveDown():
```

(3 分)

~ 全卷完 ~

試算表

常數	TRUE, FALSE
運算符	+, -, *, /, <, >, =, <>, <=, >=, &
函數	INT, RAND, SQRT, ROUND, AND, NOT, OR, LEFT, LEN, MID, RIGHT, AVERAGE, COUNT, COUNTIF, MAX, MIN, RANK, SUM, SUMIF, FIND, XLOOKUP, IF

數據庫 (SQL 指令—建基於 SQL-92 標準)

常數	FALSE, TRUE
運算符	+, -, *, /, >, <, =, >=, <=, <>, %, _, ', AND, NOT, OR
SQL	ABSOLUTE (ABS), AVG, INT, MAX, MIN, SUM, COUNT ASC, AT, CHAR (CHR), CHAR_LENGTH (LEN), LOWER, TRIM, SPACE, SUBSTRING (SUBSTR/MID), UPPER, VALUE (VAL) DATE, DAY, MONTH, YEAR ADD, ALL, ALTER, ANY, AS, ASC, BETWEEN, BY, CREATE, DELETE, DESC, DISTINCT, DROP, EXISTS, FROM, GROUP, HAVING, IN, INDEX, INNER JOIN, INSERT, INTEGER, INTERSECT, INTO, LEFT [OUTER] JOIN, LIKE, MINUS, NULL, RIGHT [OUTER] JOIN, FULL [OUTER] JOIN, ON, ORDER, SELECT, SET, TABLE, TO, UNION, UNIQUE, UPDATE, VALUES, VIEW, WHERE

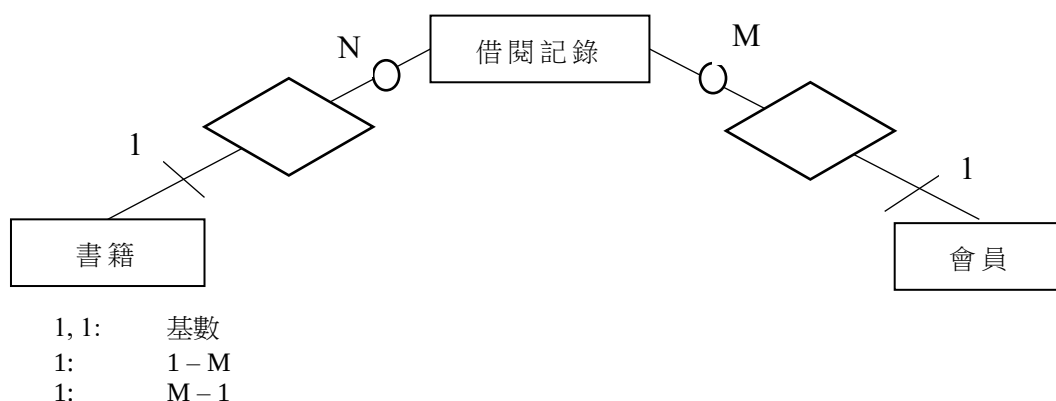
實體關係圖所採用的符號

意思	符號	意思	符號
實體		一對一關係	
屬性		一對多關係	
主要屬性		多對多關係	
關係		參與限制： 在強制參與一面用 在選擇性參與一面用	

甲部答案：

- 1 (a) (i) S2017076Maths 1
(ii) =XLOOKUP (\$A2&C\$1,Marks!\$F:\$F,Marks!\$E:\$E) 3
1 – Marks!
1 – A2&C1
1 – All correct
- (b) =SUM(B2:D2) 2
- (c) 欄：SUBJECT 1
列：STUID 1
值：MARK 的 總和 1

2.



3. Customer (CustomerID, Name)
外鍵碼：N/A
Order (OrderID, CustomerID, Date)
外鍵碼：CustomerID
Product (PID, Name, Category, Price);
外鍵碼：N/A
OrderDetails (OrderID, PID, Quantity, Total);
外鍵碼：OrderID

- 1 – 其中一個實體的屬性、主關鍵碼及名鍵碼正確 / 其中一個實體的屬性正確
1 – 其中兩個實體的屬性、主關鍵碼及名鍵碼正確 / 所有主關鍵碼正確
1 – 其中三個實體的屬性、主關鍵碼及名鍵碼正確 / 所有外鍵碼正確
1 – 全部正確

4. (a) CREATE TABLE STUDENTS (
SID VARCHAR(10) PRIMARY KEY,
NAME VARCHAR(50),
DOB DATE,
AGE INT
);
- 1 – 其中一個屬性及數據類型正確
1 – 全部正確
- (b) INSERT INTO STUDENTS (SID, NAME, DOB, AGE)
VALUES ('S001', 'Mary', '2008-01-15', 16);
- 1 – 其中一個屬性及值正確
1 – 全部正確

- (c) 使用回滾 (Rollback) 操作撤銷錯誤的交易。通過回滾操作，撤銷錯誤輸入的數據變更，恢復數據庫到操作前的狀態，確保數據的一致性和準確性。 1+1

- (d) (i) AGE 是根據 DOB 動態計算得出的屬性，其值隨著時間而變化，因此屬於衍生屬性。 1
- (ii) 其中一項： 1
- 快速查詢：AGE 被存儲後，無需每次查詢時進行計算，提升數據檢索效率。
- 便於統計：可以直接對 AGE 進行篩選和統計操作，例如分組統計某年齡段的學生數量。
- 數據簡單：如果業務邏輯中經常用到 AGE，存儲可以簡化操作，減少頻繁計算的工作量。
- 或其他合理答案。
5. (a) SELECT DOCTOR.DoctorID, COUNT(*) 1
FROM DOCTOR
INNER JOIN APPOINTMENT ON DOCTOR.DoctorID = APPOINTMENT.DoctorID
GROUP BY DOCTOR.DoctorID 1
ORDER BY TotalAppointments DESC; 1
- (b) SELECT PATIENT.PatientID, MAX(APPOINTMENT.Date) 1
FROM PATIENT
INNER JOIN APPOINTMENT ON PATIENT.PatientID = APPOINTMENT.PatientID 1
WHERE PATIENT.Age > 60 1
GROUP BY PATIENT.PatientID; 1
- (c) SELECT Name AS DoctorName 1
FROM DOCTOR
WHERE DoctorID NOT IN (SELECT DoctorID FROM APPOINTMENT); 1+1
- (d) CREATE VIEW DoctorPatientSummary AS 1
SELECT DOCTOR.Name AS DoctorName, DOCTOR.Specialty, COUNT(DISTINCT 1
PATIENT.PatientID) AS TotalPatients
FROM DOCTOR
LEFT JOIN APPOINTMENT ON DOCTOR.DoctorID = APPOINTMENT.DoctorID
LEFT JOIN PATIENT ON APPOINTMENT.PatientID = PATIENT.PatientID 1
GROUP BY DOCTOR.Name, DOCTOR.Specialty; 1
- (e) SELECT Name AS PersonName, 'Doctor' AS Role 1
FROM DOCTOR
UNION 1
SELECT Name AS PersonName, 'Patient' AS Role 1
FROM PATIENT;

乙部答案：

1. (a) 1
- | | | | | | | |
|-----|------|--|--|--|--|------|
| Q1: | Head | | | | | TAIL |
| | | | | | | |
- | | | | | | | |
|-----|------|---|----|----|--|------|
| Q2: | Head | | | | | TAIL |
| | 3 | 9 | 12 | 15 | | |
- (b) 傳回 dequeue(Q1) 1
enqueue(Q2, dequeue(Q1)) 1
enqueue(Q1, K) 1
enqueue(Q1, dequeue(Q2)) 1

2. (a) (i) 4 1
(ii) 0 1
- (b)
- | |
|-------------------|
| Push(ch) |
| Pop() |
| Pop() |
| Num2 + Num1 |
| Push(Num2 - Num1) |
| Push(Num2 * Num1) |
| Push(Num2 / Num1) |
- 6
3. (a) $A[I] = B[J]$ 2
PUSH(C, A[I]) or PUSH(C, B[J])
- (b) (i) 可以利用二分檢索法，更快尋找有沒有重複整數 1
(ii) $LEN(B) - 1$ 4
First <= Last
B[Middle] == A[I]
B[Middle] < A[I]
- (iii) 排序陣列 A 會更有效率。 2
先排序陣列 A 比較次數： $64 + 1024 * \log 16 = 4160$
先排序陣列 B 比較次數： $10240 + 16 * \log 1024 = 10400$
4. (a)
- | | | | |
|--|--|---|----|
| | | | 4 |
| | | | 4 |
| | | | 16 |
| | | 8 | 16 |
- 1
- (b) board[i][j] == 0 1
board[i][j] = 2 1
board[i][j] = 4
- (c) (i) board[i][j] > 0 1
board[i][j] 1
- (ii) board[i][j] == board[i][j + 1] 1
board[i][j] *= 2 1
- (d) (i) reverse() 1
reverse() 1
- (ii) transpose() 1
transpose() 1
- (iii) transpose() 1
moveRight() 1
transpose() 1